

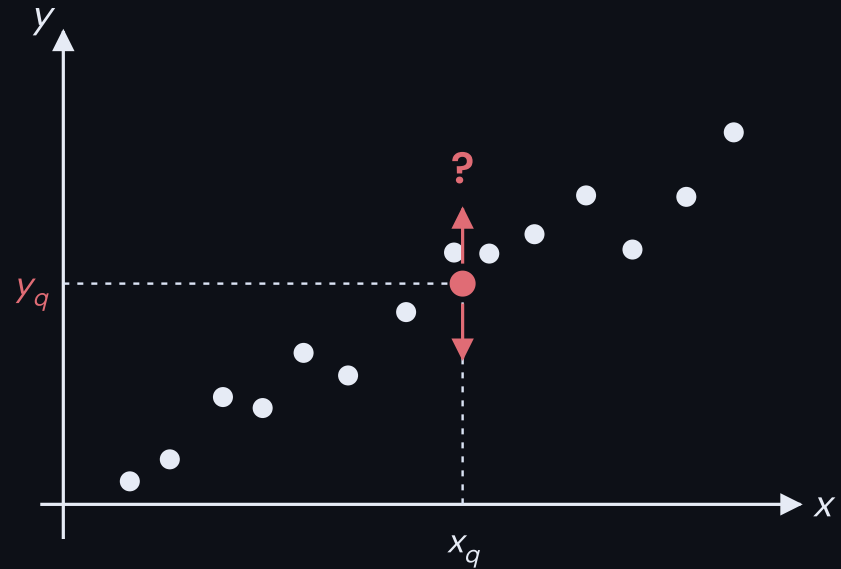
INTRODUCTION TO MACHINE LEARNING

Classification & Logistic Regression

From labels to probabilities: the sigmoid, cross-entropy, a boundary that learns before your eyes, why accuracy lies, and softmax for many classes.

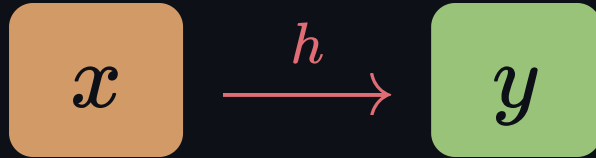
Given new input, what's the output?

Input	Output
$x^{(1)}$	$y^{(1)}$
$x^{(2)}$	$y^{(2)}$
$x^{(3)}$	$y^{(3)}$
$x^{(4)}$	$y^{(4)}$
$\vdots$	$\vdots$
Query x_q	?? Prediction



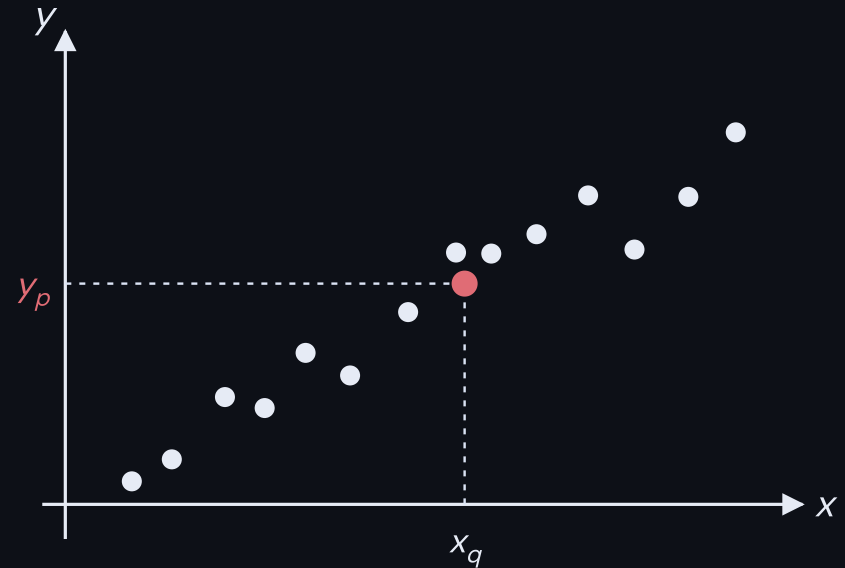
Given new input, what's the output?

Assuming $y \in \mathbb{R}$

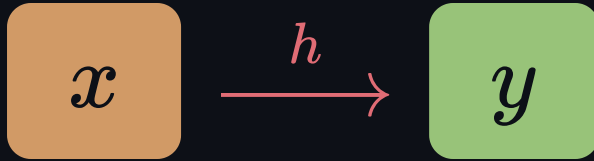


Given the data,
find a **function** h ,
that predicts y , given x

$$y = h(\mathbf{x})$$



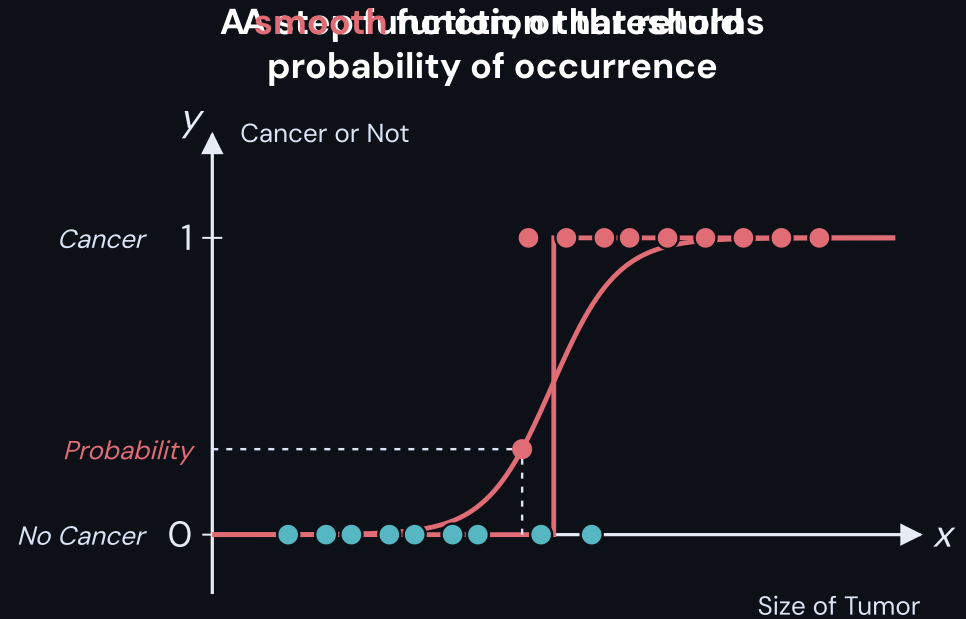
What if y is a label?



Given the data,
find a **function** h ,
that predicts y , given x

$$y = h(\mathbf{x})$$

$$y \in [0, 1]$$

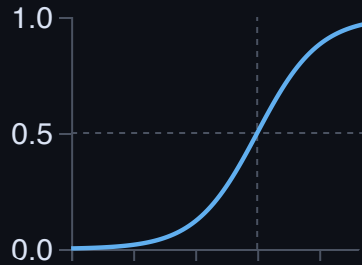


What if y is a label?

$$y = h_{\theta}(x) \text{ \& } y \in [0, 1]$$

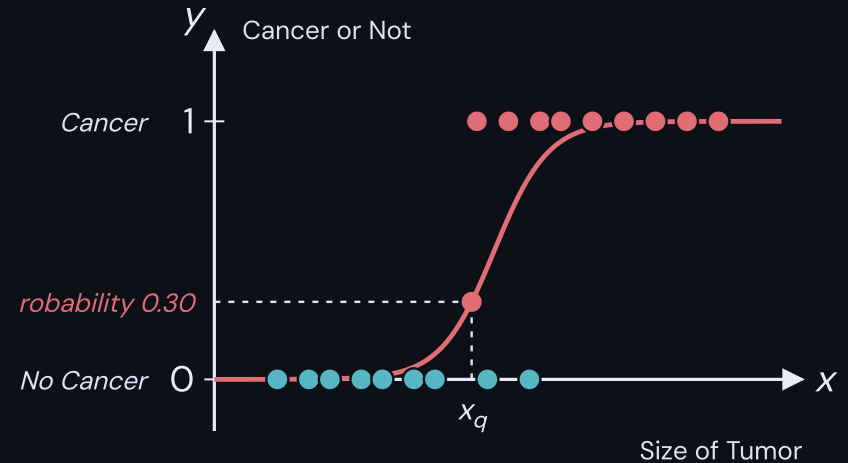
$$y = \frac{1}{1 + e^{-x}}$$

Logistic function



$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x)}}$$

A **smooth** function that returns probability of occurrence



θ_0  -10.4 θ_1  3.9

$$\text{midpoint } x_0 = -\theta_0 / \theta_1 = 2.67$$

The curve crosses $\frac{1}{2}$ where $\theta_0 + \theta_1 x = 0$, at $x_0 = -\theta_0 / \theta_1$. So θ_1 sets the steepness but also moves the midpoint; to steepen in place, scale θ_0 with it.

What if y is a label?

$$y = h_{\theta}(x) \quad \& \quad y \in [0, 1]$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta^{\top} x)}}$$

For $x \in \mathbb{R}^n$

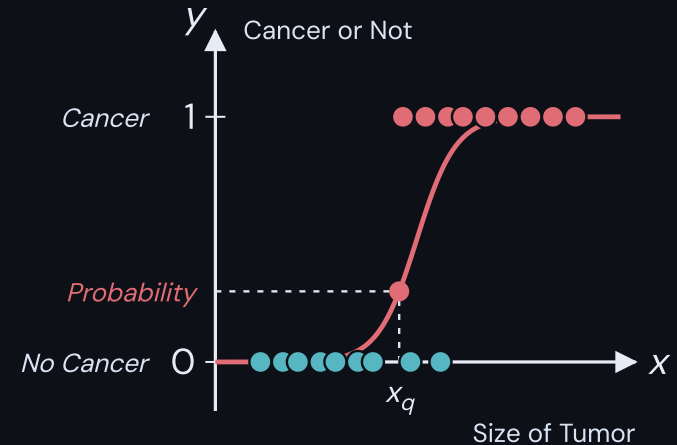
Where $\theta^{\top} x = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots$

$$\theta = [\theta_0, \theta_1, \dots]$$

$$x = [x_0, x_1, \dots]$$

Read $h_{\theta}(x)$ as $P(y = 1 \mid x; \theta)$. Still a linear model at heart: the linearity lives inside $\theta^{\top} x$; the sigmoid only turns that score into a probability.

A **smooth** function that returns probability of occurrence



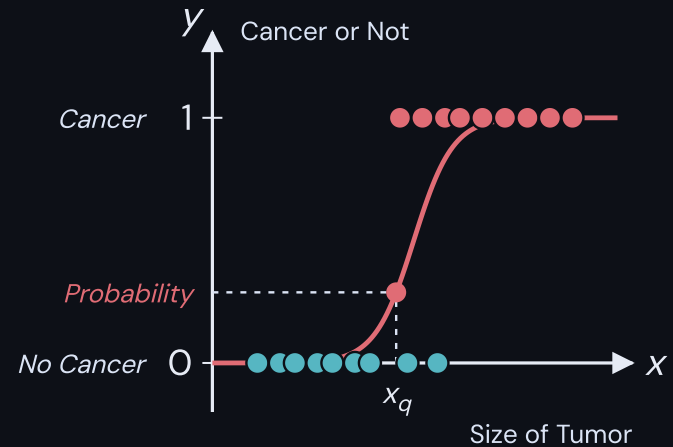
What if y is a label?

$$y = h_{\theta}(x) \quad \& \quad y \in [0, 1]$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta^{\top} x)}}$$

1. **Define a predictor:** the logistic function ✓
2. **Define a loss:** distance between function and data ?
3. **Optimize loss**
4. **Test model**

The same four steps for every supervised learning algorithm.



How do we pick the best parameters θ ?

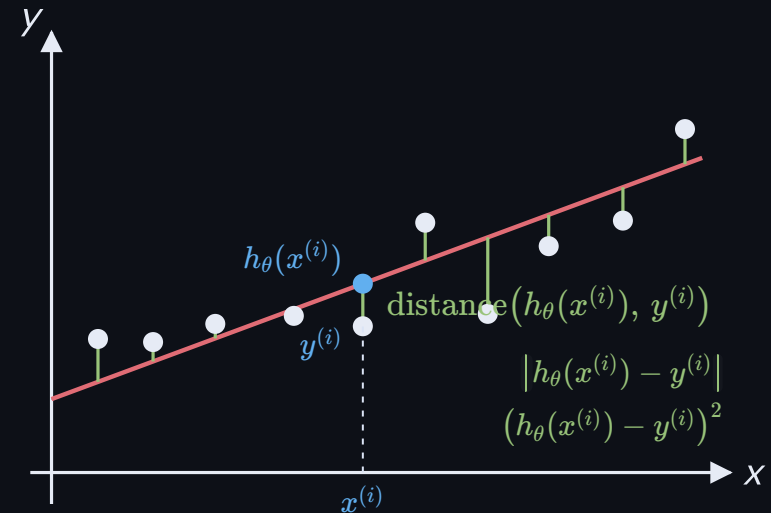
$$h_{\theta}(\mathbf{x}) = \theta^{\top} \mathbf{x} = \sum_{i=0}^d \theta_i x_i$$

Cost function

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

$$= \frac{1}{2} \sum_{i=1}^m \left(\theta^{\top} x^{(i)} - y^{(i)} \right)^2$$

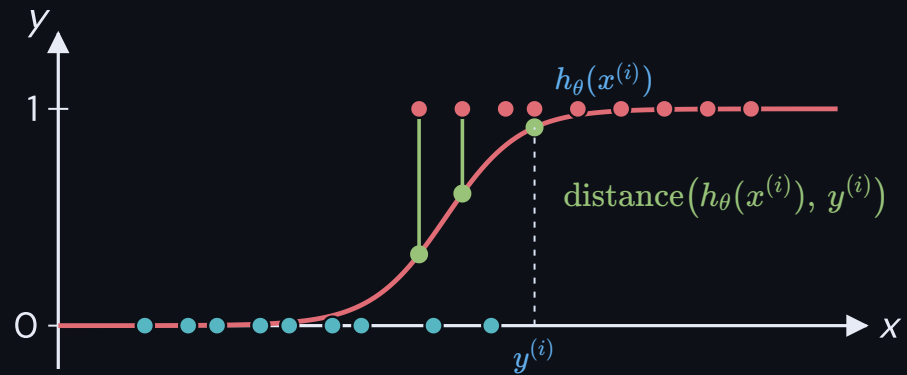
Ordinary least squares



Logistic Regression

$$y = h_{\theta}(x)$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta^{\top} x)}} = \sigma(\theta^{\top} x)$$



Linear predictor
negative log-likelihood or OLS

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Logistic predictor
Binary cross-entropy loss

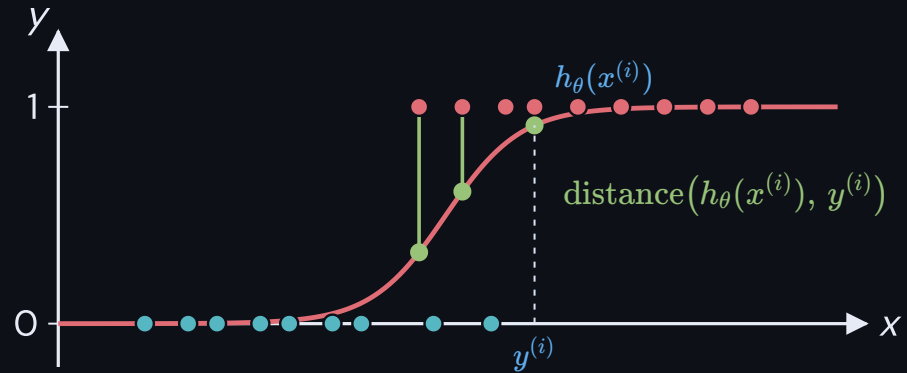
$$J(\theta) = - \sum_{i=1}^m [y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)}))]$$

Compute gradient $\nabla J(\theta)$
Gradient descent. Done!

Logistic Regression

$$y = h_{\theta}(x)$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta^{\top} x)}} = \sigma(\theta^{\top} x)$$



Linear predictor
negative log-likelihood or OLS

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

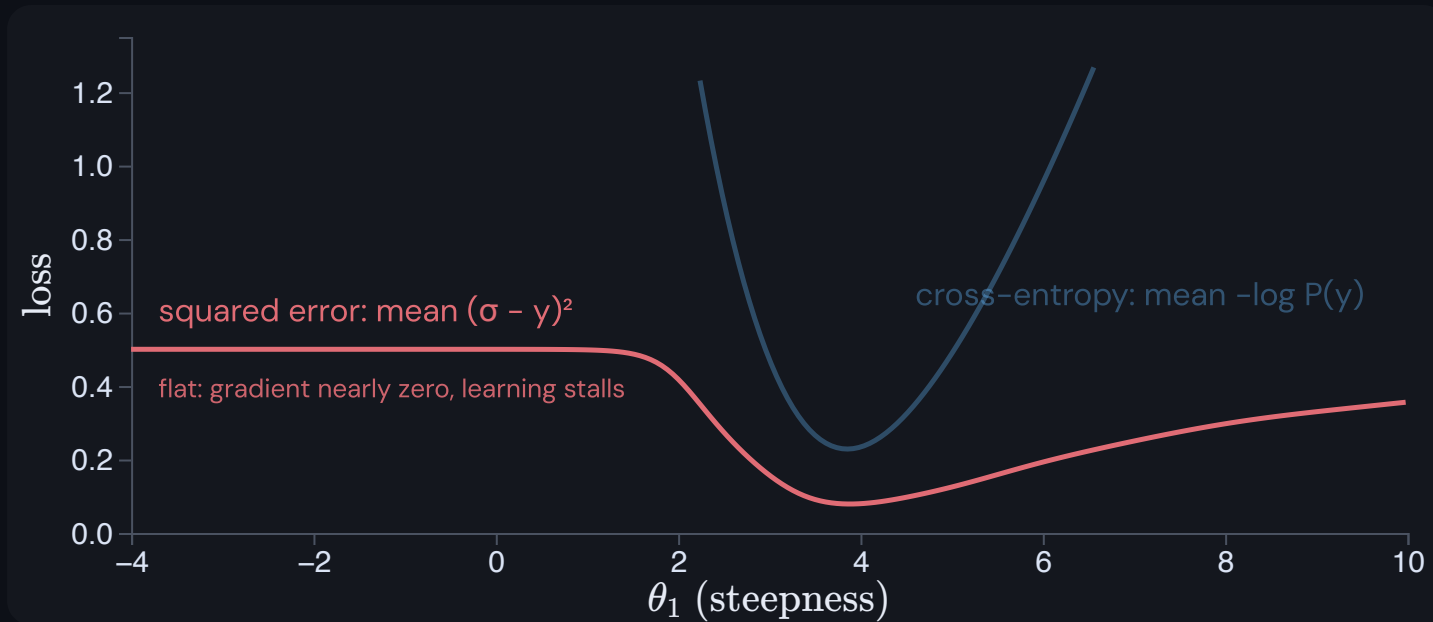
Why not use an ordinary least squares loss?

EXERCISE • DO IT NOW

What goes wrong if you fit the labels 0/1 by least squares?

```
theta = np.linalg.lstsq(X, y)[0] # why not?
```

Squared Error Meets the Sigmoid



Squared error

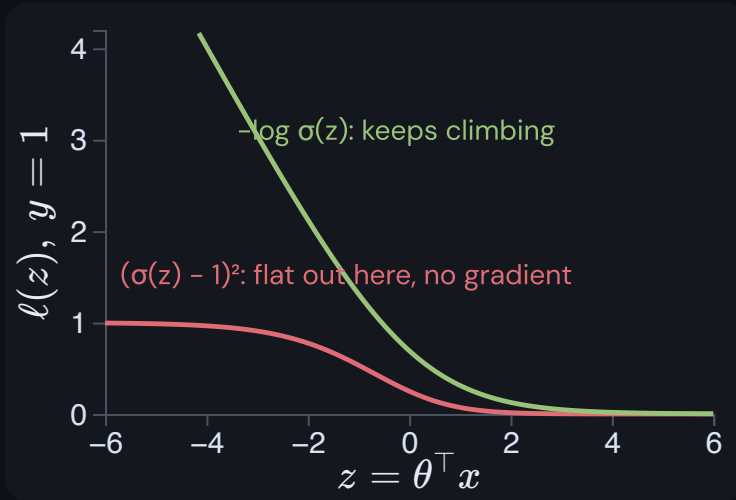
Cross-entropy

Both

The loss as a function of θ_1 on the tumor data. Squared error saturates into flat plateaus where the gradient dies; cross-entropy stays a clean bowl.

What Are Those Two Curves?

Write $z = \theta^\top x$. Each example pays a per-example loss $\ell(z)$, and $J(\theta) = \frac{1}{m} \sum_i \ell(\theta^\top x^{(i)})$ is their average: the curves on the last slide, swept over θ_1 .



One example with $y = 1$, loss against the score z .

SQUARED ERROR

$$\ell_{\text{SE}} = (\sigma(z) - y)^2$$

$$\ell'_{\text{SE}}(z) = 2(\sigma(z) - y)\sigma(z)(1 - \sigma(z))$$

Confidently wrong means $\sigma \approx 0$ or 1 , and the factor $\sigma(1 - \sigma)$ kills the gradient. Not convex in θ .

CROSS-ENTROPY

$$\ell_{\text{CE}} = -[y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))]$$

$$\ell'_{\text{CE}}(z) = \sigma(z) - y$$

The $\sigma(1 - \sigma)$ cancels against the log's derivative, so the gradient is the error itself and never dies. Convex in θ .

The Loss Probability Chooses

Same move as last lecture: the model gives a probability for each label, so ask which θ makes the observed labels most likely.

$$P(y \mid x; \theta) = \begin{cases} \sigma(\theta^\top x) & y = 1 \\ 1 - \sigma(\theta^\top x) & y = 0 \end{cases}$$

Both cases in one expression (check $y = 1$ and $y = 0$), then a product over the **i.i.d.** examples. That product is the **likelihood**:

LIKELIHOOD $L(\theta)$

$$L(\theta) = \prod_{i=1}^m P(y^{(i)} \mid x^{(i)}; \theta) = \prod_{i=1}^m \sigma(\theta^\top x^{(i)})^{y^{(i)}} (1 - \sigma(\theta^\top x^{(i)}))^{1-y^{(i)}}$$

The probability the model gives to the labels we actually observed, read as a function of θ with the data fixed. **Maximum likelihood** picks the θ that makes it largest.

Log, Then Average

Take log: the product becomes a sum and the exponents come down.

$$\log L(\theta) = \sum_{i=1}^m \left[y^{(i)} \log \sigma(\theta^\top x^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(\theta^\top x^{(i)})) \right]$$

Flip the sign to get something to minimize, and average. That is the loss J ; each example contributes its own negative log-likelihood $\ell^{(i)}$:

$$J(\theta) = -\frac{1}{m} \log L(\theta) = \frac{1}{m} \sum_{i=1}^m \ell^{(i)}(\theta), \quad \ell^{(i)} = -\log P(y^{(i)} \mid x^{(i)}; \theta)$$

Notation for the rest of the course: L is the likelihood of the whole data set, $\log L$ the log-likelihood, $\ell^{(i)}$ one example's loss, J the loss we minimize.

Binary Cross-Entropy

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \sigma(\theta^\top x^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\theta^\top x^{(i)})) \right]$$

READING IT

Only one term is active per example.
Confident and right: loss near 0.
Confident and wrong: $-\log(p)$ explodes.

WHY IT WINS

Convex in θ , no dead plateaus, and it is exactly maximum likelihood for the Bernoulli model.

Your Turn: The Gradient

You have the ingredients. Same task as linear regression, new loss.

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \sigma(\theta^\top x^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(\theta^\top x^{(i)})) \right]$$

► EXERCISE • DO IT NOW

$$\frac{\partial J}{\partial \theta_j} = ?$$

Hint: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, the derivative comes free. Take five minutes, then compare.

The Gradient Is an Old Friend

The sigmoid's self-derivative cancels the log's denominator, and everything collapses:

GRADIENT

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m \left(\sigma(\theta^\top x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

DEJA VU

Identical in form to linear regression's gradient! Different model, and different loss, but same update rule. Is this a coincidence?

So training is unchanged: $\theta := \theta - \frac{\alpha}{m} X^\top (\sigma(X\theta) - y)$.

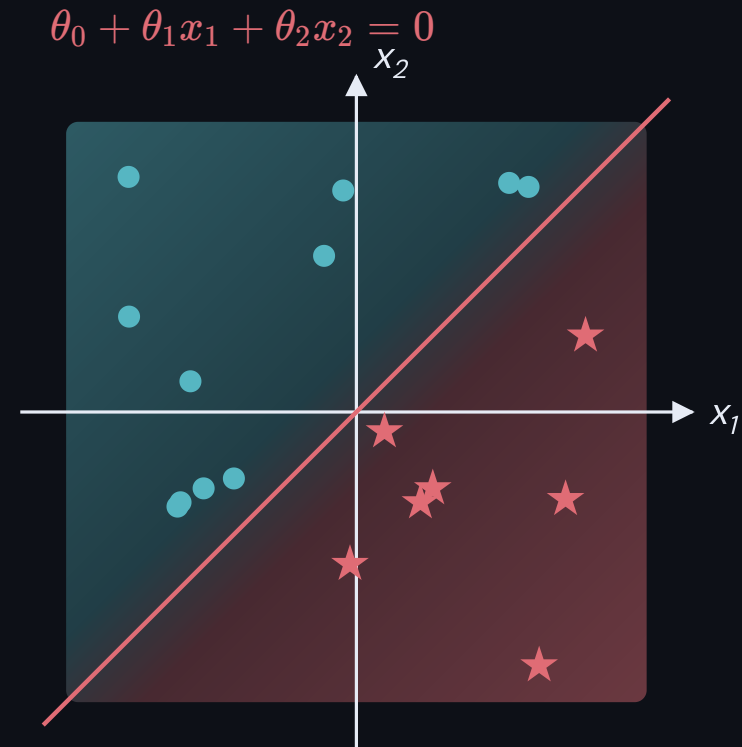
Higher dimensional inputs

For Logistic Regression and beyond

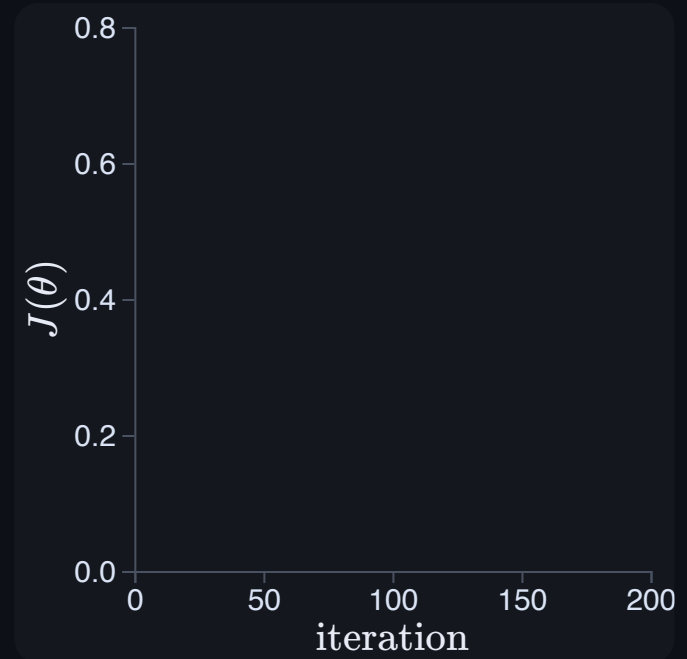
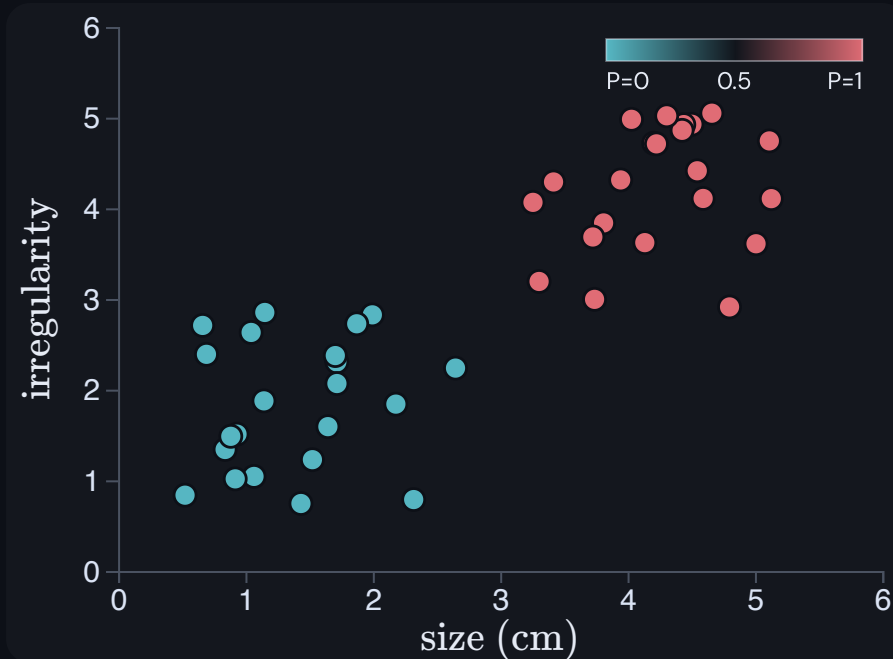
x_1	x_2	y
-2	-1	●
3	1	★
2	3	●
1	-1	★

⋮

$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2)}}$$



Watch the Boundary Learn



Step

▶ Play

Reset

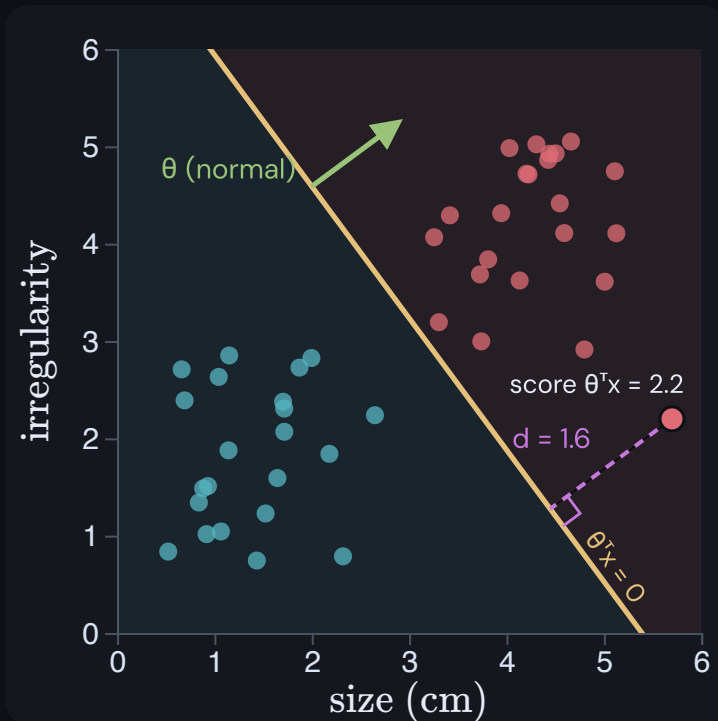
Probability shading: on

iter 0

· J = 0.693

Two measured features (tumor size, cell irregularity). The decision boundary is $\theta^\top x = 0$ and the shading is $P(y = 1)$. Click inside the plot to add a malignant point and watch the boundary respond.

The Geometry of the Boundary



THE BOUNDARY

Predict 1 when $P \geq \frac{1}{2}$, that is when the **score** $\theta_0 + \theta_1 x_1 + \theta_2 x_2$ is positive. The boundary is where the score is 0.

THE NORMAL

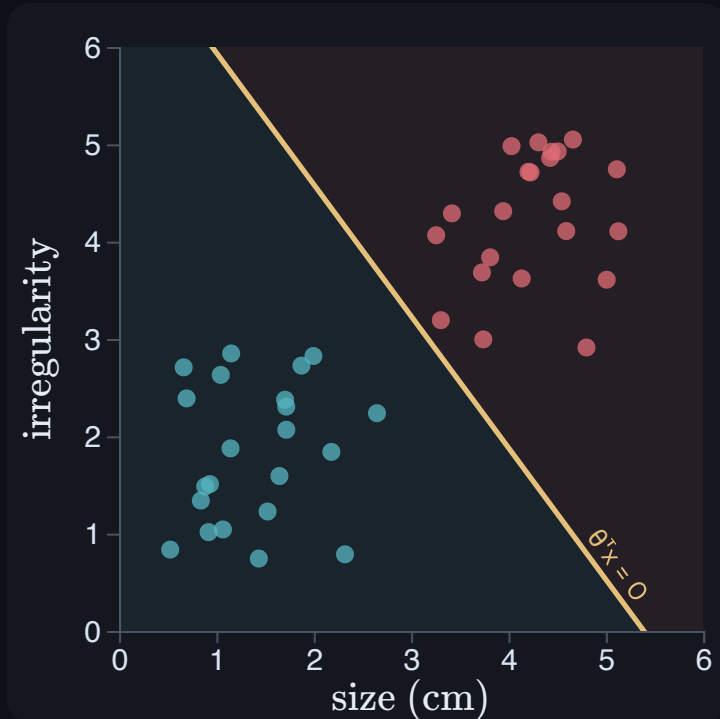
(θ_1, θ_2) is perpendicular to the line and points to the positive side. θ_0 has no direction: it only slides the line.

THE DISTANCE

$$d = \frac{\theta_0 + \theta_1 x_1 + \theta_2 x_2}{\sqrt{\theta_1^2 + \theta_2^2}}$$

θ_0 is not in the norm because it is not part of the direction. Further out, more confident:
 $P \rightarrow 0$ or 1 .

The Margin



LABELS $Y \in \{-1, +1\}$

Relabel the classes ± 1 . Then a point is on the correct side exactly when $y (\theta^T x) > 0$: the sign of the score agrees with the label.

MARGIN OF ONE POINT

$$\hat{\gamma}^{(i)} = y^{(i)} \theta^T x^{(i)} \quad \text{functional}$$

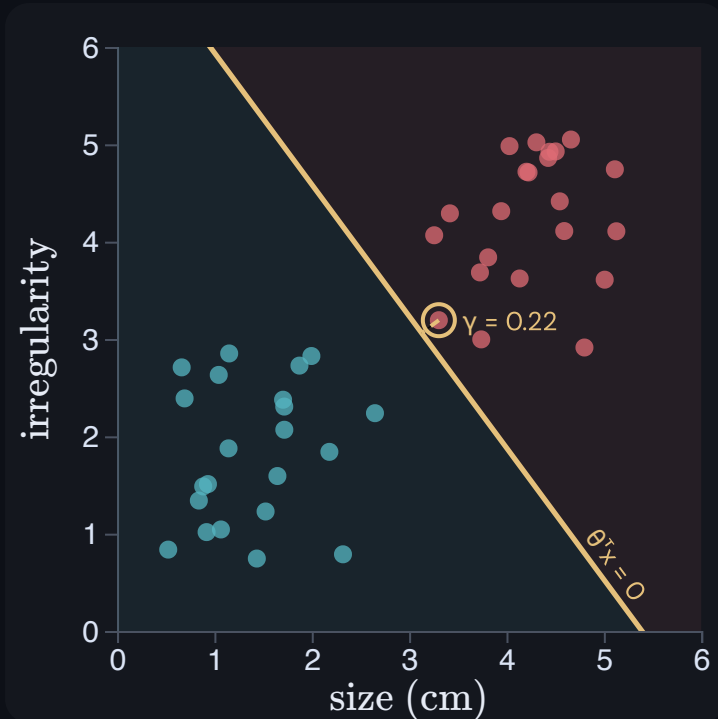
$$\gamma^{(i)} = y^{(i)} \frac{\theta^T x^{(i)}}{\|\theta_{1:2}\|} \quad \text{geometric}$$

The geometric margin is the signed distance of the last slide: positive for correct points, negative for mistakes.

MARGIN OF THE BOUNDARY

$\gamma = \min_i \gamma^{(i)}$: the distance to the closest point. Wider is safer; the boundary that maximises it is the support vector machine.

The Margin



LABELS $Y \in \{-1, +1\}$

Relabel the classes ± 1 . Then a point is on the correct side exactly when $y (\theta^T x) > 0$: the sign of the score agrees with the label.

MARGIN OF ONE POINT

$$\hat{\gamma}^{(i)} = y^{(i)} \theta^T x^{(i)} \quad \text{functional}$$

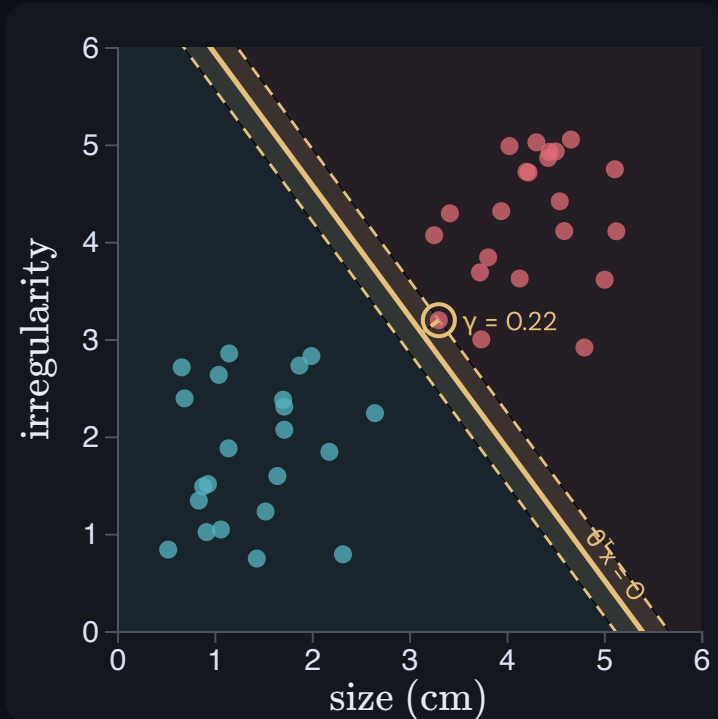
$$\gamma^{(i)} = y^{(i)} \frac{\theta^T x^{(i)}}{\|\theta_{1:2}\|} \quad \text{geometric}$$

The geometric margin is the signed distance of the last slide: positive for correct points, negative for mistakes.

MARGIN OF THE BOUNDARY

$\gamma = \min_i \gamma^{(i)}$: the distance to the closest point. Wider is safer; the boundary that maximises it is the support vector machine.

The Margin



LABELS $Y \in \{-1, +1\}$

Relabel the classes ± 1 . Then a point is on the correct side exactly when $y (\theta^T x) > 0$: the sign of the score agrees with the label.

MARGIN OF ONE POINT

$$\hat{\gamma}^{(i)} = y^{(i)} \theta^T x^{(i)} \quad \text{functional}$$

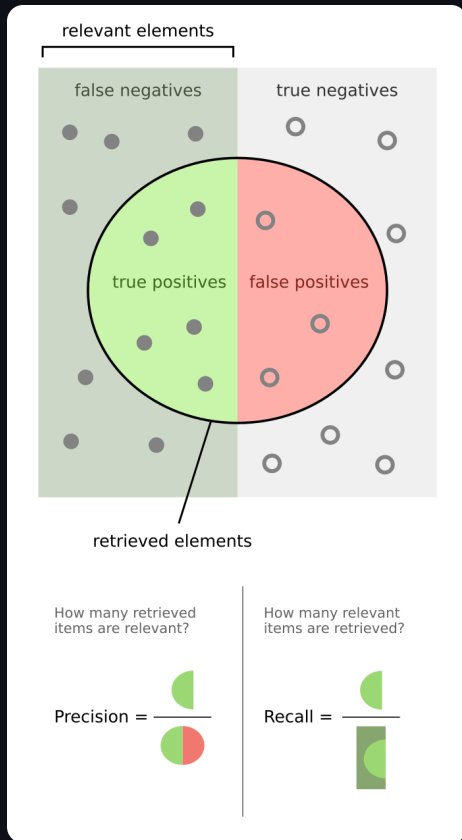
$$\gamma^{(i)} = y^{(i)} \frac{\theta^T x^{(i)}}{\|\theta_{1:2}\|} \quad \text{geometric}$$

The geometric margin is the signed distance of the last slide: positive for correct points, negative for mistakes.

MARGIN OF THE BOUNDARY

$\gamma = \min_i \gamma^{(i)}$: the distance to the closest point. Wider is safer; the boundary that maximises it is the support vector machine.

Step 4: Test. But Accuracy of What?



Every prediction lands in one of four cells: **TP** flagged and real, **FP** flagged and not, **FN** missed, **TN** correctly ignored.

PRECISION

$$P = \frac{TP}{TP + FP}$$

Of everything we **flagged**, how much was real.

ACCURACY

$$\frac{TP + TN}{TP + TN + FP + FN}$$

Of everything, how much we got right.

RECALL

$$R = \frac{TP}{TP + FN}$$

Of everything **real**, how much we caught.

F1

$$F_1 = \frac{2PR}{P + R}$$

One number when you need one: the harmonic mean of P and R.

The Same Four Numbers on One Screening

1000 people screened, 50 of them ill. The classifier flags 100: 40 who are ill and 60 who are not.

	pred ill	pred well
ill (50)	40	10
well (950)	60	890

accuracy $930 / 1000 = 0.93$

precision $40 / 100 = 0.40$

recall $40 / 50 = 0.80$

F1 0.53

THE TRAP

Flag nobody: accuracy 0.95, better than the classifier, and recall 0. Accuracy rewards agreeing with the majority.

WHICH NUMBER, WHEN

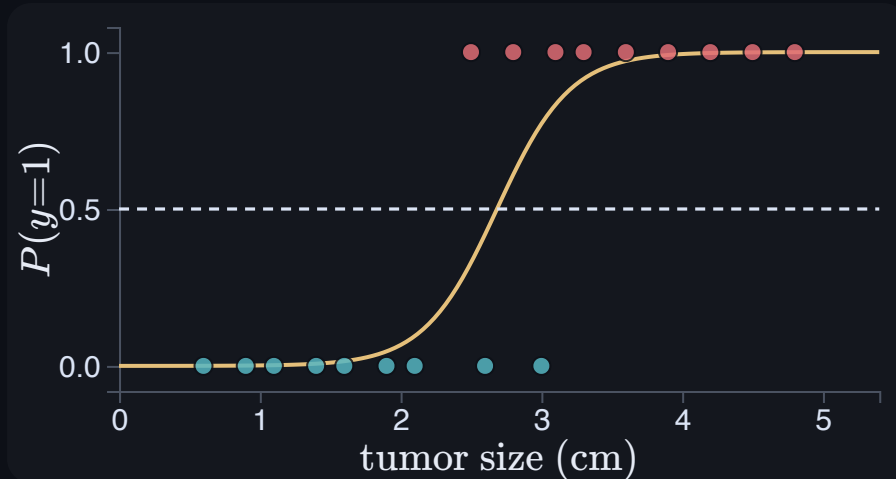
Recall when a miss is the expensive error: cancer screening, fraud, a security scan.

Precision when a false alarm is the expensive error: spam filters (a real email lost), an alert a human must investigate.

Accuracy only when classes are balanced and both errors cost the same.

F1 when you must rank models by one number and care about both.

When Accuracy Lies: Imbalanced Data



threshold 0.50

Rare-disease mode: off

	pred 1	pred 0
true 1	8	1
true 0	1	8

accuracy 0.89

precision 0.89

recall 0.89

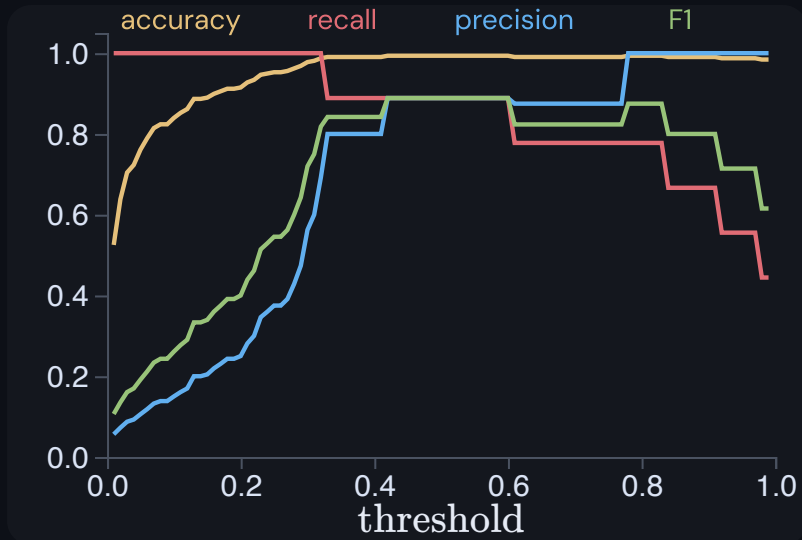
F1 0.89

Lower the threshold to catch more disease (recall up, precision down). There is no free lunch, only a chosen trade.

PRACTICAL ADVICE

With 1% positives, predicting "no" for everyone scores 99% accuracy and catches nobody. Report precision and recall, not accuracy. Then **move the threshold**: lower it to buy recall when a miss is expensive (disease, fraud), raise it to buy precision when a false alarm is expensive.

Reading the Metrics Together



Tumour data with 300 more healthy patients (6% ill): one trained model, every threshold from 0 to 1.

ONE MODEL, MANY CLASSIFIERS

Training fixes θ ; the threshold is chosen afterwards. Lower it: recall up, precision down. Raise it: the reverse.

WHAT ACCURACY IS DOING

Past a threshold of 0.3 it sits above 0.95 and barely moves, while precision and recall swing between 0.5 and 1. It cannot see the trade, so it is the wrong compass here.

WHAT TO REPORT

Confusion matrix, precision and recall at the threshold you deploy. To compare models regardless of threshold: the precision-recall curve, or the ROC curve, and the area under it.

Logistic Regression in Twelve Lines

```
1 import numpy as np
2
3 def sigmoid(z):
4     return 1.0 / (1.0 + np.exp(-z))
5
6 X = np.c_[np.ones(len(x1)), x1, x2]  # n x 3: bias + two features
7 theta = np.zeros(3)
8 alpha = 0.5
9
10 for t in range(2000):                # batch gradient descent
11     p = sigmoid(X @ theta)           # predicted probabilities
12     theta -= alpha / len(y) * (X.T @ (p - y))
13
14 print(theta)                          # boundary: theta . x = 0
```

Line 12 is the whole lecture: (prediction error) times (input), averaged. Swap the sigmoid for the identity and this is last week's linear regression code. In scikit-learn the same fit is

`LogisticRegression().fit(X, y).`

EXERCISE • DO IT NOW

What is the loop doing: batch gradient descent, stochastic gradient descent, something else? What would you change to make it the other one?

Logistic Regression in Twelve Lines

```
1 import numpy as np
2
3 def sigmoid(z):
4     return 1.0 / (1.0 + np.exp(-z))
5
6 X = np.c_[np.ones(len(x1)), x1, x2]  # n x 3: bias + two features
7 theta = np.zeros(3)
8 alpha = 0.5
9
10 for t in range(2000):                # batch gradient descent
11     p = sigmoid(X @ theta)           # predicted probabilities
12     theta -= alpha / len(y) * (X.T @ (p - y))
13
14 print(theta)                          # boundary: theta . x = 0
```

Line 12 is the whole lecture: (prediction error) times (input), averaged. Swap the sigmoid for the identity and this is last week's linear regression code. In scikit-learn the same fit is

`LogisticRegression().fit(X, y).`

EXERCISE • DO IT NOW

What is the loop doing: batch gradient descent, stochastic gradient descent, something else? What would you change to make it the other one?

Logistic Regression in Twelve Lines

```
1 import numpy as np
2
3 def sigmoid(z):
4     return 1.0 / (1.0 + np.exp(-z))
5
6 X = np.c_[np.ones(len(x1)), x1, x2]  # n x 3: bias + two features
7 theta = np.zeros(3)
8 alpha = 0.5
9
10 for t in range(2000):                # batch gradient descent
11     p = sigmoid(X @ theta)           # predicted probabilities
12     theta -= alpha / len(y) * (X.T @ (p - y))
13
14 print(theta)                         # boundary: theta . x = 0
```

Line 12 is the whole lecture: (prediction error) times (input), averaged. Swap the sigmoid for the identity and this is last week's linear regression code. In scikit-learn the same fit is

`LogisticRegression().fit(X, y).`

EXERCISE • DO IT NOW

What is the loop doing: batch gradient descent, stochastic gradient descent, something else? What would you change to make it the other one?

Logistic Regression in Twelve Lines

```
1 import numpy as np
2
3 def sigmoid(z):
4     return 1.0 / (1.0 + np.exp(-z))
5
6 X = np.c_[np.ones(len(x1)), x1, x2]  # n x 3: bias + two features
7 theta = np.zeros(3)
8 alpha = 0.5
9
10 for t in range(2000):                # batch gradient descent
11     p = sigmoid(X @ theta)           # predicted probabilities
12     theta -= alpha / len(y) * (X.T @ (p - y))
13
14 print(theta)                          # boundary: theta . x = 0
```

Line 12 is the whole lecture: (prediction error) times (input), averaged. Swap the sigmoid for the identity and this is last week's linear regression code. In scikit-learn the same fit is

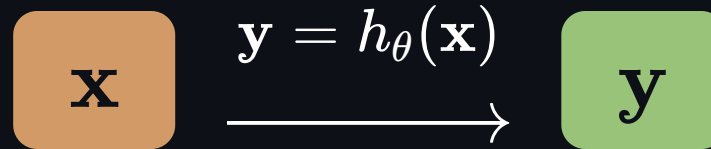
`LogisticRegression().fit(X, y).`

EXERCISE • DO IT NOW

What is the loop doing: batch gradient descent, stochastic gradient descent, something else? What would you change to make it the other one?

Higher dimensional outputs

Multi-class classification: **Softmax** regression



k discrete values for representing output $\mathbf{y}$


$$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

car


$$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

plane


$$\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

boat


$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

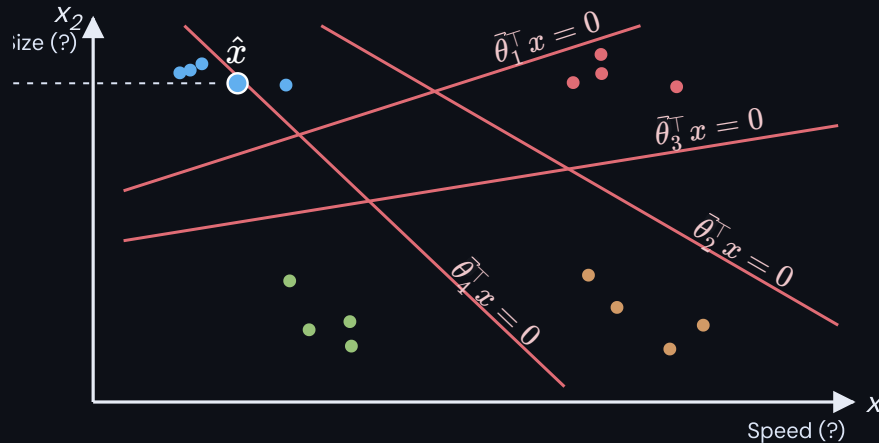
horse

One-hot encoding

How to turn scores into probabilities?

Note: this slide explains the intuition. Softmax regression is not exactly equivalent to a combination of one-vs-all classifiers.

What's the probability it's a car?



WARNING!!!
Notation Alert
 $\vec{\theta}_i$ is a vector

Score

$$\begin{aligned}\vec{\theta}_1^\top \hat{x} &= 3 \\ \vec{\theta}_2^\top \hat{x} &= -0.3 \\ \vec{\theta}_3^\top \hat{x} &= -0.8 \\ \vec{\theta}_4^\top \hat{x} &= -22\end{aligned}$$

→
exp

Positive measure

$$\begin{aligned}\exp(3) &= 20.1 \\ \exp(-0.3) &= 0.74 \\ \exp(-0.8) &= 0.45 \\ \exp(-22) &= 0.00 \dots 1\end{aligned}$$

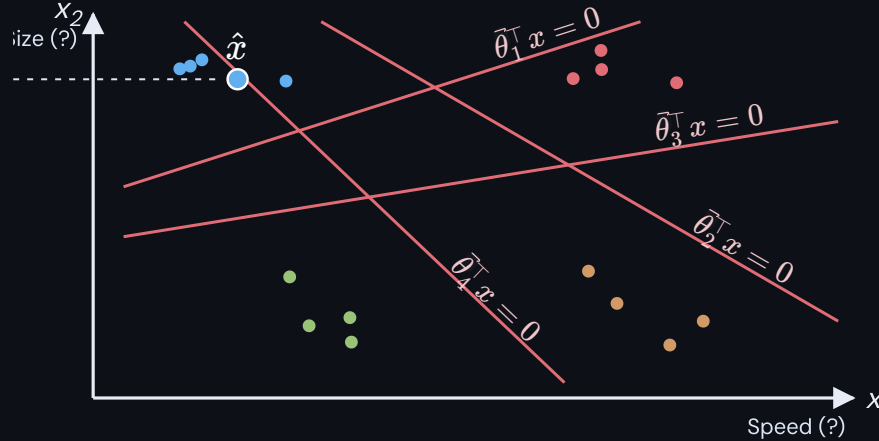
→
Normalize

Softmax

$$\hat{p}(y = i \mid x; \theta) = \frac{\exp(\vec{\theta}_i^\top x)}{\sum_{j=1}^k \exp(\vec{\theta}_j^\top x)}$$

How to turn scores into probabilities?

What's the probability it's a car?



WARNING!!!
Notation Alert
 $\vec{\theta}_i$ is a vector

Softmax

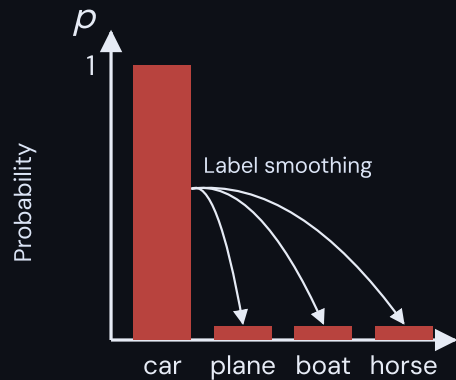
$$\hat{p}(y = i \mid x; \theta) = \frac{\exp(\vec{\theta}_i^T x)}{\sum_{j=1}^k \exp(\vec{\theta}_j^T x)}$$

$$\hat{p}_{\Theta} = \begin{bmatrix} \hat{p}(y = \text{car} \mid x; \theta_1) \\ \hat{p}(y = \text{plane} \mid x; \theta_2) \\ \hat{p}(y = \text{boat} \mid x; \theta_3) \\ \hat{p}(y = \text{horse} \mid x; \theta_4) \end{bmatrix}$$

$$\hat{p}_{\Theta} = \text{Softmax}(\Theta x)$$

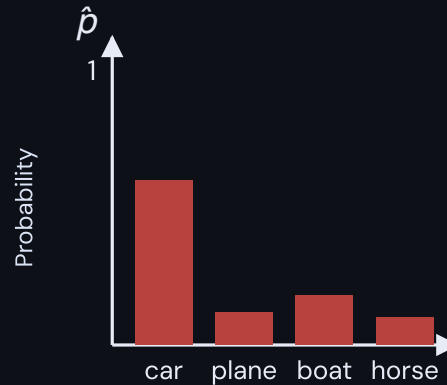
How do you train?

Given label



$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$
car

Prediction

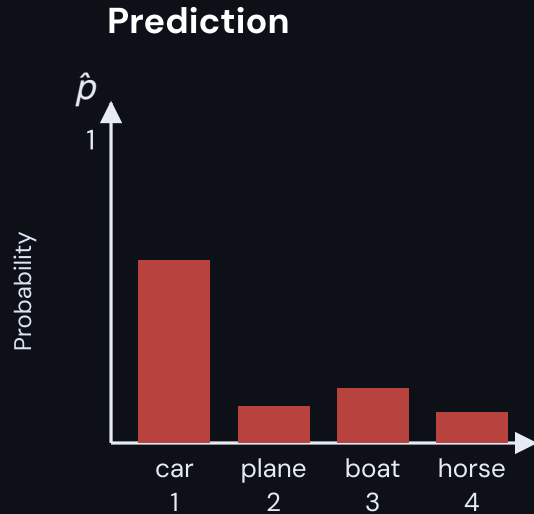


WARNING!!!
 y is the
ground-truth index

$\operatorname{argmin}_{\vec{\theta}_1, \vec{\theta}_2, \vec{\theta}_3, \vec{\theta}_4}$

$$\begin{aligned} \text{CrossEntropy}(p, \hat{p}) &= - \sum_{i=1}^k p(y = i) \log(\hat{p}(y = i)) \\ &= - \log(\hat{p}(y = 1)) \end{aligned}$$

How do you train?



$$\text{CrossEntropy}(p, \hat{p}) = - \sum_{i=1}^k p(y = i) \log(\hat{p}(y = i))$$

$$\begin{aligned} \text{Logit} &= -\log(\hat{p}(y = 1)) \\ &= -\log\left(\frac{\exp(\vec{\theta}_y^\top x)}{\sum_{j=1}^k \exp(\vec{\theta}_j^\top x)}\right) \end{aligned}$$

The logit is the score $\vec{\theta}_i^\top x$ inside the exponential; the softmax turns logits into probabilities.

Train with Gradient Descent!

Explore Further

Some useful reference interactive websites:

[MLU-Explain: Logistic Regression \(visual essay\)](#)

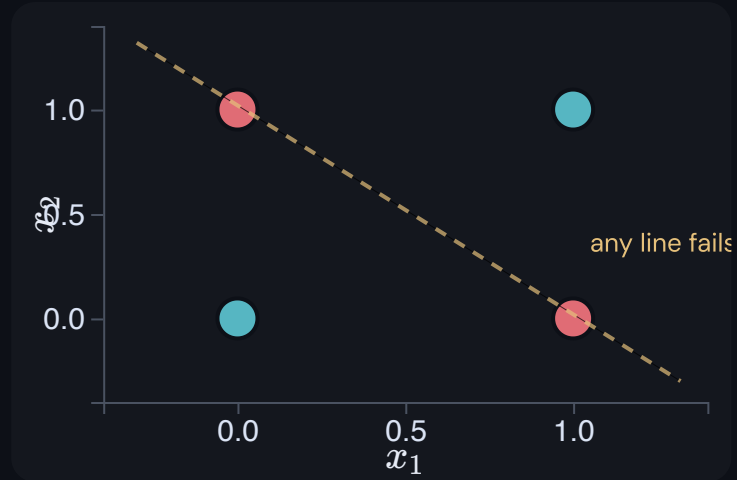
[TensorFlow Playground: a single sigmoid neuron](#)

They open inside the slide; press **Esc** to come back.

LOOK CLOSELY

The Playground link has zero hidden layers and a sigmoid output. That configuration *is* logistic regression: one neuron. Hold that thought for two lectures.

What if the boundary needs to *curve*?



No straight line separates these classes. Two answers exist: invent better features by hand, or let the model learn them. That second answer is called a **neural network**.

Summary

THE MODEL

$$P(y=1 \mid x) = h_{\theta}(x) = \sigma(\theta^{\top} x)$$

a linear score squashed to a probability

THE LOSS

$$J(\theta) = -\frac{1}{m} \sum_i [y \log \sigma + (1-y) \log(1-\sigma)]$$

maximum likelihood for Bernoulli labels;
convex

THE UPDATE

$$\theta := \theta - \frac{\alpha}{m} X^{\top} (\sigma(X\theta) - y)$$

error times input, again

MANY CLASSES

$$\hat{p}_{\Theta} = \text{Softmax}(\Theta x)$$

one-hot labels, one score per class,
cross-entropy loss

Caveats: boundaries are straight; accuracy misleads when classes are imbalanced; pick the threshold for the cost you care about. Next: maximum likelihood as a worldview, and the family that unites linear and logistic regression.